

Date: _____

Talha Abdullah

SU92-BAIFM-F25-002

BAI - 3C

Assignment 1

Data Structure

Date: _____

Question No1

```
class Queue:
    limit
    def __init__(self, value):
        self.queue = []
        self.limit = value limit

    def enqueue(self, value):
        if len(self.queue) == self.limit:
            print("Queue is full")
        else:
            self.queue.append(value)
            print("Updated queue: ", self.queue)

    def dequeue(self):
        if not self.queue:
            print("No jobs to process")
        else:
            job = self.queue.pop(0)
            print("Processed job: ", job)
            print("Updated Queue: ", self.queue)

    def peek(self):
        if not self.queue:
            print("No jobs waiting")
        else:
            print("Next job to process: ", self.queue[0])
```

SUFI

Date: _____

```
class Stack:
```

```
    def __init__(self, limit):
```

```
        self.stack = []
```

```
        self.limit = limit
```

```
    def push(self, value):
```

```
        if len(self.stack) == self.limit:
```

```
            print("Stack is full")
```

```
        else:
```

```
            self.stack.append(value)
```

```
            print("Updated stack: ", self.stack)
```

```
    def pop(self):
```

```
        if not self.stack:
```

```
            print("No revision to undo")
```

```
        else:
```

```
            removed = self.stack.pop()
```

```
            print("Updated Revision: ", removed)
```

```
            print("Updated stack: ", stack self.stack)
```

```
    def peek(self):
```

```
        if not self.stack:
```

```
            print("Revision history is empty")
```

```
        else:
```

```
            print("Current Revisions: ", self.stack[-1])
```

SUFI

Date: _____

```
def show-history(self):  
    if not self.stack:  
        print("Revision history is empty")  
    else:  
        print("Revision history:", self.stack)
```

```
stack-size = int(input("Enter stack size (for revisions): "))  
queue-size = int(input("Enter queue size (for print jobs): "))  
stack = Stack(stack-size)  
queue = Queue(queue-size)
```

while True:

```
    print("In--- Document Management System---")  
    print("1. Push new document revision")  
    print("2. Undo latest revision")  
    print("3. Peek current revision")  
    print("4. Show full revision history")  
    print("5. Enqueue new print job")  
    print("6. peek next print job")  
    print("7. sort and process print jobs")  
    print("8. Exit")  
    choice = input("Enter choice: ")
```

if choice == "1":

```
    rev = input("Enter revision discription: ")  
    stack.push(rev)
```

SUFI

Date: _____

```
elif choice == "2":
    stack.pop()
elif choice == "3":
    stack.peak()
elif choice == "4":
    stack.show-history()
elif choice == "5":
    job = int(input("Enter print job priority: "))
    queue.enqueue(job)
elif choice == "6":
    queue.peak()
elif choice == "7":
    if not queue.queue:
        print("No jobs to process")
    else:
        print("In Sorting print jobs: ")
        bubble_sort(queue.queue)
        print("Final sorted Queue: ", queue.queue)

    print("In Processing Jobs: ")
    while queue.queue:
        queue.dequeue()
elif choice == "8":
    print("Program is exiting")
    break
else:
    print("Invalid choice. Try again")
```

Date: _____

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n-1):  
        swapped = False  
        for j in range(n-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
                swapped = True  
        print(f" After pass {i+1}: ", arr)  
        if not swapped:  
            break
```

Question No 2

```
class Stack:  
    def __init__(self, limit):  
        self.stack = []  
        self.limit = limit  
  
    def push(self, value):  
        if len(self.stack) == self.limit:  
            print("Stack is full")  
        else:  
            self.stack.append(value)  
            print("Session added: ", value)
```

SUFI

Date: _____

```
def pop(self):  
    if not self.stack:  
        print("No session to cancel.")  
    else:  
        removed = self.stack.pop()  
        print("Canceled session: ", removed)  
        print("Current Agenda: ", self.stack)
```

```
def peek(self):  
    if not self.stack:  
        print("Agenda is empty.")  
    else:  
        print("Next session: ", self.stack[-1])
```

```
def is-empty(self):  
    return len(self.stack) == 0
```

```
def show-agenda(self):  
    print("Full agenda: ", self.stack)
```

class Queue:

```
    def __init__(self, limit):  
        self.queue = []  
        self.limit = limit
```

Date: _____

```
def enqueue(self, attendee):  
    if len(self.queue) == self.limit:  
        print("Queue is full")  
    else:  
        self.queue.append(attendee)  
        print("Attendee added: ", attendee)
```

```
def dequeue(self):  
    if not self.queue:  
        print("No attendees waiting.")  
    else:  
        admitted = self.queue.pop(0)  
        print("Admitted attendee:", admitted)  
        print("Queue size now:", self.size())
```

```
def peek(self):  
    if not self.queue:  
        print("No attendees waiting")  
    else:  
        print("Next attendee: ", self.queue[0])
```

```
def size(self):  
    return len(self.queue)
```

```
def bubble_sort_desc(arr):  
    n = len(arr)
```

Date: _____

```
for i in range(n-1):
    swapped = False
    for j in range(n-i-1):
        if arr[j][1] < arr[j+1][1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
            swapped = True
    print(f" After pass {i+1} : {arr}")
    if not swapped:
        break
```

```
s_size = int(input("Enter agenda size (stack capacity): "))
q_size = int(input("Enter attendee queue size: "))
stack = Stack(s_size)
queue = Queue(q_size)
```

while True:

```
print("\n --- Event Planning menu --- ")
print("1. Add session to agenda")
print("2. Cancel latest session")
print("3. Peek next session")
print("4. Show full agenda")
print("5. Add attendee")
print("6. Peek next attendee")
print("7. Sort and admit attendees")
print("8. Exit")
choice = input("Enter choice: ")
```

SUFI

Date: _____

```
if choice == "1":
    session = input("Enter session: ")
    stack.push(session)
elif choice == "2":
    stack.pop()
elif choice == "3":
    stack.peak()
elif choice == "4":
    stack.show-agenda()
elif choice == "5":
    name = input("Enter attendee name: ")
    vip = int(input("Enter vip level: "))
    queue.enqueue(name, vip)
elif choice == "6":
    queue.peak()
elif choice == "7":
    if not queue.queue:
        print("No attendees waiting.")
    else:
        print("In Sorting attendees by VIP:")
        bubble_sort_desc(queue.queue)
        print("Final sorted Queue:", queue.queue)
    print("In Admitting Attendees: ")
    while queue.queue:
        queue.dequeue()
```

Date: _____

```
elif choice == "8":
```

```
    print("Exiting the program")
```

```
    break
```

```
else:
```

```
    print("Invalid choice. Try again")
```